DARSHAN UNIVERSITY

Department of Computer Science & Engineering

Course Code: 2601CS102

Web Designing

B.Tech. First Semester

CHAPTER 3

CSS3 — Modern Styling & Interactivity

E-Notes / Study Material (Print-Friendly Edition)

Topic	Coverage
Introduction to CSS3	What's new in CSS3, module-based structure
Transitions, Transforms & Animations	transition, transform, @keyframes, animation properties
Tool-tips	Pure CSS hover tooltips, positioning & arrows
Styling Images	Filters, hover effects, object-fit, shapes
Media Queries	Responsive breakpoints, mobile-first design
Wildcard Selectors	Universal & attribute wildcard matching
Variables	CSS Custom Properties (--var), theming
Gradients	Linear, radial & conic gradients
Pseudo Class & Elements	State-based & content-generating selectors

Academic Year 2025–26

1. Introduction to CSS3

1.1 What is New in CSS3?

CSS3 is the latest major evolution of CSS. Unlike CSS2, which was released as one single specification, CSS3 was split into independent modules (Selectors, Backgrounds, Animations, Flexbox, Grid, etc.) — each developed, updated, and adopted by browsers at its own pace. This modular approach is why CSS3 features roll out gradually rather than all at once.

Major Features Introduced in CSS3:

Rounded corners (border-radius) — no more sliced background-image hacks
Shadows (box-shadow, text-shadow) — depth without images
Gradients (linear-gradient, radial-gradient) — no more gradient image files
Transitions & Animations — smooth motion without JavaScript
Flexbox & Grid — real layout systems, replacing float-based hacks
Media Queries — fully responsive, device-adaptive design
Custom Properties (CSS Variables) — reusable, dynamic design tokens
Transforms — rotate, scale, skew, translate elements in 2D and 3D

1.2 Why CSS3 Matters

Before CSS3	With CSS3
Rounded corners needed image slices	border-radius: 10px; — one line
Shadows required extra background images	box-shadow: 2px 2px 5px gray;
Animations needed JavaScript/Flash	Pure CSS @keyframes animations
Layouts used floats & hacks	Flexbox & Grid — real layout tools
Fixed, non-adaptive designs	Fully responsive via media queries

📌 **Browser Support Note:**

Almost all CSS3 features are fully supported in modern browsers (Chrome, Firefox, Safari, Edge).
Always test on multiple browsers for university/college lab submissions, since older systems may run outdated browser versions.

2. Transitions, Transforms & Animations

CSS3 provides three related techniques for visual motion and transformation: Transitions (smooth A-to-B changes), Transforms (change an element's position, size, angle or shape), and Animations (full multi-step, timeline-based sequences using @keyframes). Transforms are often combined with transitions or animations to create smooth visual effects.

2.1 Transitions — Simple A-to-B Animation

A transition smoothly changes a CSS property's value from its current state to a new state over a specified duration. It is commonly triggered by :hover, :focus, or a class/state change. A transition does not define multiple timeline steps; it interpolates between the starting and ending values.

```
.btn {
  background-color: teal;
  transition: background-color 0.3s ease, transform 0.3s ease;
}

.btn:hover {
  background-color: navy;
  transform: scale(1.05);
}
```

Property	Purpose	Example Value
transition-property	Which property to animate	background-color, all
transition-duration	How long the animation takes	0.3s, 500ms
transition-timing-function	The speed curve of the animation	ease, linear, ease-in-out
transition-delay	Wait time before animation starts	0.2s

2.2 Transform — Move, Rotate, Scale & Skew

The transform property changes the visual position, size, rotation or shape of an element without changing the normal document flow. Transforms are commonly combined with transitions to make hover effects smooth.
Common transform functions:

Function	Purpose	Example
translate(x, y)	Moves an element horizontally and/or vertically	transform: translate(20px, 10px);
translateX(x)	Moves along the X-axis	transform: translateX(30px);
translateY(y)	Moves along the Y-axis	transform: translateY(20px);
rotate(angle)	Rotates an element	transform: rotate(15deg);
scale(x, y)	Changes visual size	transform: scale(1.1);
skew(x, y)	Tilts/shears an element	transform: skew(10deg, 5deg);

Example — transform with transition:

```
.card {  transition: transform 0.3s ease; }  .card:hover {  transform: translateY(-8px) scale(1.03); }
```

Important points:
• translate() changes visual position.

• rotate() changes the angle.
• scale() increases or decreases visual size.
• skew() slants the element.
• transform-origin controls the point around which a transform is performed.
• transform by itself is immediate; transition or animation is used when a smooth change is required.
• Transforms normally do not change the space occupied by the element in the document layout.
transform-origin example:

```
.box {   transform-origin: top left;   transform: rotate(20deg); }
```

2D and 3D transforms:

Type	Examples
2D	translate(), rotate(), scale(), skew()
3D	translateZ(), rotateX(), rotateY(), rotateZ(), scale3d()

Transition vs Transform vs Animation:

Feature	Transition	Transform	Animation
Main purpose	Smooth change between two states	Change position, size, rotation or shape	Multi-step motion over time
Typical trigger	State change such as :hover	Applied directly or combined with transition/animation	Can start automatically or be triggered
Timeline	Start → End	Not a timeline by itself	Multiple keyframes such as 0%, 50%, 100%
Example	transition: color 0.3s;	transform: scale(1.1);	@keyframes slideIn { ... }

2.3 @keyframes — Multi-Step Animation

For animations with multiple stages (not just A-to-B), use @keyframes to define exact steps, then apply them with the animation property.

```
@keyframes slideIn {
  0%   { transform: translateX(-100%); opacity: 0; }
  50%  { opacity: 0.5; }
  100% { transform: translateX(0); opacity: 1; }
}

.card {
  animation-name: slideIn;
  animation-duration: 1s;
  animation-timing-function: ease-out;
  animation-iteration-count: 1;
  animation-fill-mode: forwards;
}

/* Shorthand — same result in one line */
.card {
  animation: slideIn 1s ease-out forwards;
}
```

Property	Purpose
animation-name	Name of the @keyframes rule to use
animation-duration	How long ONE cycle of the animation takes
animation-iteration-count	How many times it repeats — number or infinite
animation-direction	normal, reverse, alternate (bounces back and forth)

animation-fill-mode	forwards keeps the final keyframe state after finishing
animation-play-state	running or paused — can be toggled with JS/hover

Transition vs Animation — Key Difference:

Transition = A simple A → B change, needs a trigger like :hover or a class change.

Animation = A full timeline with multiple steps (0%, 50%, 100%), can run automatically on page load, and can loop infinitely.

3. Tool-tips

A tooltip is a small popup box that displays extra information when a user hovers over an element. Tooltips can be built entirely with CSS — no JavaScript required — using the :hover pseudo-class combined with positioning.

3.1 Building a Pure CSS Tooltip

```
<div class="tooltip-container">
  Hover Me
  <span class="tooltip-text">This is a helpful tooltip!</span>
</div>

<style>
.tooltip-container {
  position: relative;   /* establishes positioning context for the tooltip */
  display: inline-block;
  border-bottom: 1px dotted gray;
  cursor: help;
}a

.tooltip-text {
  visibility: hidden;
  opacity: 0;
  width: 160px;
  background-color: #1e293b;
  color: white;
  text-align: center;
  padding: 6px 10px;
  border-radius: 6px;

  position: absolute;
  bottom: 125%;         /* places tooltip above the trigger element */
  left: 50%;
  transform: translateX(-50%);

  transition: opacity 0.3s;
}

/* Show tooltip smoothly on hover */
.tooltip-container:hover .tooltip-text {
  visibility: visible;
  opacity: 1;
}
</style>
```

3.2 Adding an Arrow to the Tooltip

A small triangle 'arrow' is created using the ::after pseudo-element with clever border tricks — no image needed.

```
.tooltip-text::after {
  content: "";
  position: absolute;
  top: 100%;              /* places arrow at the bottom of the tooltip box */
  left: 50%;
  margin-left: -5px;
```

```
  border-width: 5px;
  border-style: solid;
  border-color: #1e293b transparent transparent transparent;
}
```

Positioning Value	Tooltip Appears
bottom: 125%; (on tooltip)	Above the trigger element
top: 125%; (on tooltip)	Below the trigger element
right: 110%; top: 50%;	To the left of the trigger element
left: 110%; top: 50%;	To the right of the trigger element

Key Requirement:

The parent container MUST have position: relative — this is what the tooltip's position: absolute anchors to. Without it, the tooltip would position itself relative to the entire page instead of the hovered element.

4. Styling Images

CSS3 provides powerful properties to transform, filter, and creatively present images without needing to edit the actual image file in Photoshop.

4.1 Shape & Border Effects

```
/* Rounded corners */
img.rounded { border-radius: 12px; }

/* Perfect circle — great for profile pictures */
img.avatar {
  width: 100px;
  height: 100px;
  border-radius: 50%;
  object-fit: cover;   /* prevents stretching/distortion */
}

/* Polaroid-style frame */
img.polaroid {
  border: 10px solid white;
  box-shadow: 0 4px 10px rgba(0,0,0,0.3);
}
```

4.2 object-fit — Controlling How Images Scale

When an image doesn't perfectly match its container's aspect ratio, object-fit controls how it's cropped or scaled to fit.

```
img {
  width: 300px;
  height: 200px;
  object-fit: cover;    /* fills container, crops overflow — most common */
}
```

Value	Behavior
fill	Stretches image to fill the box exactly (may distort)
cover	Fills the box completely, cropping overflow (keeps aspect ratio)
contain	Fits the whole image inside the box (may leave empty space)
none	Image keeps its natural size, ignoring the container

4.3 Filters & Hover Effects

```
/* Filters — adjust image appearance */
img.grayscale { filter: grayscale(100%); }
img.blurred   { filter: blur(4px); }
img.bright    { filter: brightness(130%); }
img.dark-mode { filter: invert(1); }
```

```
/* Smooth hover zoom effect */
.gallery img {
  transition: transform 0.4s ease, filter 0.4s ease;
}
.gallery img:hover {
  transform: scale(1.1);
  filter: brightness(110%);
}

/* Grayscale by default, full color on hover */
img.fade-color {
  filter: grayscale(100%);
  transition: filter 0.3s;
}
img.fade-color:hover {
  filter: grayscale(0%);
}
```

Filter Function	Effect
grayscale(%)	Converts image to black & white
blur(px)	Blurs the image
brightness(%)	Lightens or darkens the image
contrast(%)	Increases/decreases contrast
sepia(%)	Applies a warm vintage-brown tone
invert(%)	Inverts all colors (negative effect)

5. Media Queries

Media queries let you apply different CSS rules based on the characteristics of the device — most commonly, screen width. This is the foundation of responsive web design.

5.1 Basic Syntax

```
@media (max-width: 768px) {
  /* CSS rules here apply ONLY when screen width is 768px or less */
  body {
    font-size: 14px;
  }
  .sidebar {
    display: none;
  }
}
```

Query Type	Applies When
(max-width: 768px)	Screen width is 768px OR SMALLER
(min-width: 768px)	Screen width is 768px OR LARGER
(min-width: 768px) and (max-width: 1024px)	Width is BETWEEN 768px and 1024px
(orientation: portrait)	Device is taller than it is wide

5.2 Common Responsive Breakpoints

```
/* Mobile devices */
@media (max-width: 576px) {
  .container { padding: 10px; }
}

/* Tablets */
@media (min-width: 577px) and (max-width: 992px) {
  .container { padding: 20px; }
}

/* Laptops & Desktops */
@media (min-width: 993px) {
  .container { padding: 40px; }
}
```

5.3 Mobile-First vs Desktop-First Approach

Approach	Strategy	Uses
Mobile-First	Write base CSS for mobile, then ADD styles for larger screens	min-width queries
Desktop-First	Write base CSS for desktop, then OVERRIDE for smaller screens	max-width queries

Industry Best Practice:

Mobile-First is the recommended modern approach — most web traffic today comes from mobile devices. Always include this in your HTML <head> for media queries to work correctly on phones:
<meta name="viewport" content="width=device-width, initial-scale=1.0">

6. Wildcard Selectors

Wildcard selectors let you match multiple elements based on patterns rather than exact names — extremely useful for applying broad resets or targeting groups of similar attributes.

6.1 The Universal Selector (*)

```
/* Matches EVERY single element on the page */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

/* Combined with a descendant — matches every element INSIDE .card */
.card * {
  font-family: inherit;
}
```

> **Most Common Real-World Use:**
> * { box-sizing: border-box; } is used at the top of nearly every professional stylesheet — a universal CSS reset.

6.2 Attribute Wildcard Selectors

These selectors use pattern-matching symbols (^, $, *) inside attribute selectors to match partial attribute values — effectively acting as 'wildcards' for HTML attributes.

```
/* [attr^="value"] — STARTS WITH */
a[href^="https"] { color: green; }        /* all secure links */
img[src^="icons/"] { width: 24px; }       /* all images from icons/ folder */

/* [attr$="value"] — ENDS WITH */
a[href$=".pdf"] { color: red; }           /* all links to PDF files */
img[src$=".png"] { background: #eee; }    /* all PNG images */

/* [attr*="value"] — CONTAINS ANYWHERE */
a[href*="youtube"] { color: darkred; }    /* any link containing "youtube" */
div[class*="col-"] { padding: 10px; }     /* any class containing "col-" */
```

Wildcard Symbol	Matches
^ (caret)	Attribute value STARTS WITH the given text
$ (dollar)	Attribute value ENDS WITH the given text
* (asterisk)	Attribute value CONTAINS the given text anywhere

> ⚠ **Don't Confuse Two Different Uses of *:**

* alone = Universal selector (matches ALL elements)
[attr*="value"] = Attribute contains wildcard (matches elements whose attribute CONTAINS that text)
Same symbol, completely different meaning depending on where it's used!

7. Variables (CSS Custom Properties)

CSS Variables (officially called Custom Properties) let you store reusable values — colors, sizes, fonts — in one place and reference them anywhere in your stylesheet. Changing the variable updates every usage instantly.

7.1 Declaring & Using Variables

```
:root {
  --primary-color: #0d9488;
  --secondary-color: #1e3a5f;
  --spacing-unit: 16px;
  --font-main: 'Poppins', sans-serif;
}

body {
  font-family: var(--font-main);
  color: var(--secondary-color);
}

.btn {
  background-color: var(--primary-color);
  padding: var(--spacing-unit);
}

.btn:hover {
  background-color: var(--secondary-color);
}
```

Concept	Explanation
:root	A pseudo-class representing the top of the document — the standard place to declare global variables
-- prefix	Every custom property name MUST start with a double dash
var(--name)	The function used to RETRIEVE a variable's value anywhere in the CSS
var(--name, fallback)	Provides a fallback value if the variable isn't defined

7.2 Practical Example — Dark Mode Theme Switching

Variables can be redefined inside a different selector (like a .dark-mode class) to instantly re-theme an entire page:

```
:root {
  --bg-color: white;
  --text-color: #111;
}

.dark-mode {
  --bg-color: #111;
  --text-color: white;
}
```

```
body {
  background-color: var(--bg-color);
  color: var(--text-color);
  transition: background-color 0.3s, color 0.3s;
}
<!-- Just toggle the class="dark-mode" on <body> via JavaScript to switch themes
-->
```

Why Use CSS Variables Instead of Just Retyping Values?

Change --primary-color once → updates every button, link, and heading using it automatically.

Perfect for maintaining brand consistency across a large project.

Unlike Sass variables, CSS variables can be changed live with JavaScript, enabling dynamic theme switching.

8. Gradients

CSS gradients create smooth transitions between two or more colors, directly in CSS — no image files needed. They render sharply at any screen size (fully scalable).

8.1 Linear Gradients

```
/* Basic — left to right by default */
.box1 { background: linear-gradient(red, blue); }

/* Specify direction */
.box2 { background: linear-gradient(to right, red, blue); }
.box3 { background: linear-gradient(to bottom right, red, blue); }

/* Specify exact angle */
.box4 { background: linear-gradient(45deg, red, blue); }

/* Multiple color stops */
.box5 { background: linear-gradient(to right, red, yellow, green); }

/* Precise color stop positions */
.box6 { background: linear-gradient(to right, red 20%, blue 80%); }
```

8.2 Radial Gradients

Radial gradients radiate outward from a center point in a circular or elliptical shape, instead of a straight line.

```
/* Basic radial gradient — circle from center */
.box7 { background: radial-gradient(circle, yellow, orange); }

/* Positioned off-center */
.box8 { background: radial-gradient(circle at top left, white, teal); }

/* Elliptical shape (default) */
.box9 { background: radial-gradient(ellipse, pink, purple); }
```

8.3 Conic Gradients

A conic gradient sweeps colors AROUND a center point (like a color wheel or pie chart), rather than outward or in a line.

```
.pie-chart {
  background: conic-gradient(red 0deg 90deg, blue 90deg 180deg,
                             green 180deg 270deg, yellow 270deg 360deg);
}

.color-wheel {
  background: conic-gradient(red, yellow, green, cyan, blue, magenta, red);
  border-radius: 50%;
}
```

Gradient Type	Direction of Color Flow	Common Use Case
linear-gradient()	Straight line — any angle	Buttons, backgrounds, banners
radial-gradient()	Outward from a center point	Spotlight effects, badges
conic-gradient()	Rotating around a center point	Pie charts, color wheels, loaders

Pro Tip:

Combine gradients with text: background: linear-gradient(...); + background-clip: text; + color: transparent; creates trendy gradient-colored text headings.

9. Pseudo Class and Elements

Pseudo-classes and pseudo-elements are special CSS selectors that target states or specific parts of elements that don't exist as separate tags in the HTML.

9.1 Pseudo-Classes (single colon :)

A pseudo-class targets an element based on a special STATE or its POSITION among sibling elements.

Pseudo-Class	Triggers When
:hover	Mouse pointer rests over the element
:focus	Element is clicked into or tabbed to (inputs, links)
:active	Element is being actively clicked
:visited / :link	A link already visited / not yet visited
:first-child / :last-child	First or last child of its parent
:nth-child(n)	The nth child — supports odd, even, or formulas like 3n
:checked	A checkbox or radio button currently checked
:disabled / :enabled	A form field that is disabled or enabed
:not(selector)	Matches everything EXCEPT the given selector

```
a:hover { color: red; }
input:focus { border-color: teal; outline: none; }

/* Zebra-striped table rows — no extra HTML classes needed */
tr:nth-child(even) { background: #f4f4f4; }

/* Style everything except the danger button */
button:not(.btn-danger) { background: teal; }
```

9.2 Pseudo-Elements (double colon ::)

A pseudo-element targets a specific PART of an element's content, or inserts generated content that doesn't exist anywhere in the HTML.

Pseudo-Element	Targets / Inserts
::before	Inserts generated content immediately BEFORE the element's content
::after	Inserts generated content immediately AFTER the element's content
::first-letter	Styles only the FIRST letter of a text block
::first-line	Styles only the FIRST line of a text block
::selection	Styles text while the user is highlighting/selecting it

::placeholder	Styles the placeholder text inside an empty input field

```
/* content is REQUIRED — even if empty ("") */
.required::after {
  content: " *";
  color: red;
}

/* Drop-cap effect */
p::first-letter {
  font-size: 2.5rem;
  font-weight: bold;
  color: navy;
}

::selection {
  background: teal;
  color: white;
}
```

⚠ **Pseudo-Class vs Pseudo-Element — The Core Difference:**

Pseudo-CLASS (:) → targets a STATE or POSITION of an existing element → a:hover, li:first-child

Pseudo-ELEMENT (::) → targets a PART of an element, or generates brand-new content → p::first-letter, .box::before

content: ""; is mandatory for ::before / ::after to render at all.

Quick Reference — CSS3 Cheat Sheet

Animations

Property/Rule	Purpose
transition: property duration timing;	Simple A→B animation on state change
@keyframes name { 0% {} 100% {} }	Defines multi-step animation sequence
animation: name duration timing fill-mode;	Applies a @keyframes animation to an element
transform: scale()/rotate()/translate();	Moves, rotates or resizes an element visually

Tooltips

Technique	Purpose
position: relative (parent) + position: absolute (tooltip)	Anchors tooltip to its trigger element
:hover + visibility/opacity	Shows/hides tooltip smoothly
::after with border-color trick	Creates a pointer arrow

Styling Images

Property	Purpose
border-radius: 50%	Circular image
object-fit: cover/contain	Controls image scaling within a box
filter: grayscale()/blur()/brightness()	Visual image filters

Media Queries

Query	Meaning
@media (max-width: 768px) {}	Applies at 768px or smaller
@media (min-width: 768px) {}	Applies at 768px or larger

Wildcard Selectors

Selector	Meaning
*	Universal selector — matches everything
[attr^="val"]	Attribute starts with value
[attr$="val"]	Attribute ends with value
[attr*="val"]	Attribute contains value anywhere

Variables

Syntax	Purpose
:root { --name: value; }	Declares a global CSS variable
var(--name)	Uses a variable's value
var(--name, fallback)	Uses variable with a fallback

Gradients

Function	Direction
linear-gradient(direction, colors)	Straight-line color transition
radial-gradient(shape, colors)	Outward from a center point
conic-gradient(colors)	Rotates around a center point

Pseudo-Classes & Elements

Type	Examples
Pseudo-Class (:)	:hover, :focus, :nth-child(n), :not()
Pseudo-Element (::)	::before, ::after, ::first-letter, ::selection

Practice Questions — CSS3

Instructions:

Attempt all questions. Short answer questions carry 2 marks each. Long answer / coding questions carry 5 marks each.

Section A — Short Answer Questions (2 marks each)

1. What is new in CSS3 compared to CSS2? Name any three major features.
2. Differentiate between transition and animation in CSS3.
3. What is the purpose of the @keyframes rule?
4. Why must the parent container of a CSS tooltip have position: relative?
5. What is the difference between object-fit: cover and object-fit: contain?
6. Write the media query syntax to apply styles only on screens 768px or smaller.
7. What is a wildcard selector? Give one example using the ^ symbol.
8. What is the correct syntax to declare a CSS variable and use it?
9. What is the difference between linear-gradient() and radial-gradient()?
10. What is the difference between a pseudo-class and a pseudo-element?

Section B — Long Answer Questions (5 marks each)

1. Explain CSS Animations using @keyframes. Create an animation that makes a box slide in from the left and fade in over 1 second. (5 marks)
2. Explain how to build a pure-CSS tooltip with an arrow, including all required properties. (5 marks)
3. Explain Media Queries and the Mobile-First approach. Write CSS that changes a container's padding at three different breakpoints. (5 marks)
4. Explain CSS Variables (Custom Properties) with a practical example of building a light/dark theme switcher. (5 marks)
5. Explain the three types of CSS gradients (linear, radial, conic) with a code example of each. (5 marks)
6. Explain structural and interaction pseudo-classes with examples, and explain why content: "" is required for ::before/::after. (5 marks)

11. What is the difference between transform and transition in CSS3? Give one example of each.
12. Explain translate(), rotate(), scale() and skew() with suitable CSS examples.
13. What is transform-origin? Explain with an example.

Section C — Coding Exercises

Create a card that moves upward and becomes slightly larger on hover using transition and transform. (5 marks)
Create a square that rotates 45 degrees on hover and returns smoothly to its original position when the pointer leaves. (4 marks)

1. Create a button that changes background color and slightly grows in size smoothly on hover using CSS transitions. (3 marks)

2. Build an image gallery where each image is grayscale by default and turns to full color with a zoom effect on hover. (5 marks)

3. Create a responsive 3-column layout that becomes 1 column on screens smaller than 768px using media queries. (5 marks)

4. Define 3 CSS variables for a color theme (--primary, --secondary, --bg) and use them to style a card component. (4 marks)